

SU-GPT: A Retrieval-Augmented Academic Advising System for Sabancı University Students

Korhan Erdoğan — 30838
Mehmet Selman Yılmaz — 31158
CS 455: Large Language Models
Sabancı University
Spring 2025–2026

Abstract

Academic advising requires institution-specific and version-sensitive information, including degree requirements, course classifications, prerequisites, schedules, and student history. General-purpose large language models can answer such questions fluently while relying on outdated or unsupported information. This paper presents SU-GPT, a retrieval-augmented academic advising assistant for Sabancı University. The current implementation combines a React/Vite frontend, a FastAPI backend, MongoDB for structured application state, ChromaDB for vector retrieval, MiniLM embeddings, cross-encoder reranking, and an API-hosted LLM. MongoDB is treated as the durable source of truth, while ChromaDB is a reproducible vector index.

The active local corpus uses the `su_knowledge` Chroma collection and currently contains course-oriented records only; review, exam, and syllabus-specific source documents are intentionally absent and are handled with explicit uncertainty messages rather than fabricated retrieval evidence. We implemented a 68-question grounded benchmark and a Section 5 evaluation runner that records `results.jsonl`, retrieved chunks, summary metrics, and run configuration. A measured pilot run over the first 10 benchmark questions and two retrieval modes produced Recall@6 of 0.20 for dense retrieval and 0.10 for hybrid rerank. The same run measured average end-to-end latency of 20.05 seconds for dense retrieval and 42.22 seconds for hybrid rerank. These pilot results are not a final claim for the full 68-question benchmark; they are a reproducible checkpoint and reveal concrete retrieval failures caused by near-duplicate schedule rows, short slide-title chunks, and course-code matching weaknesses.

1 Introduction

University students frequently ask questions such as which courses remain compulsory, whether a course counts as an area or free elective, who teaches a section, when a course meets, and what they should take next. Correct answers require combining official curriculum rules, course descriptions, program-specific classifications, schedules, credit constraints, and the student’s own academic history.

A general-purpose LLM cannot be expected to contain accurate and current information for every university curriculum. Even if it produces fluent prose, it may silently apply the wrong curriculum year, infer nonexistent prerequisites, or answer a schedule question from stale world knowledge. Retrieval-augmented generation (RAG) addresses this by retrieving external documents and inserting them into the generation context [1].

SU-GPT is a course-aware RAG assistant for Sabancı University. It retrieves information from course, curriculum, graduation, schedule, and course-material records; reads selected user courses from MongoDB; and returns an answer with human-readable sources and `source_chunk_ids`. The project is primarily an application/system-building project and secondarily an evaluation/analysis project.

The contributions are:

- an end-to-end academic advising application connecting React, FastAPI, MongoDB, ChromaDB, local retrieval models, and an API-hosted LLM;
- a production-oriented source-of-truth model where MongoDB stores durable source metadata and ChromaDB is a rebuildable vector index;

- deterministic source and chunk identifiers for reproducible ingestion;
- a unified retrieval-mode contract covering `llm_only`, BM25, dense, dense rerank, hybrid, and hybrid rerank modes;
- a 68-question benchmark and evaluation runner with Recall@K, MRR@K, nDCG@K, latency, token-estimate, and cost-estimate fields;
- an initial failure-analysis workflow seeded with real observed retrieval misses.

2 Background

2.1 Retrieval-Augmented Generation

RAG combines information retrieval and language generation. A retriever first identifies documents related to the question; the generator then receives those documents as context. This architecture is appropriate for institutional knowledge that changes over time or is absent from public model-training data [1].

2.2 Dense Retrieval, BM25, and Hybrid Search

Dense retrieval represents documents and queries as vectors. SU-GPT uses the compact MiniLM sentence-transformer model `all-MiniLM-L12-v2`, which is suitable for semantic search [5]. BM25 is a lexical retrieval method based on exact term matching and document statistics [7]. Because course codes such as `CS412` are exact identifiers, lexical signals are useful alongside dense similarity.

The current codebase includes BM25, dense, dense-rerank, hybrid, and hybrid-rerank modes through a shared `run_retrieval_mode` contract. Hybrid search fuses BM25 and dense candidates using reciprocal-rank fusion (RRF) [8]. Cross-encoder reranking then scores query-document pairs jointly, which can improve ordering when the gold document is already in the candidate set [6].

3 System Architecture

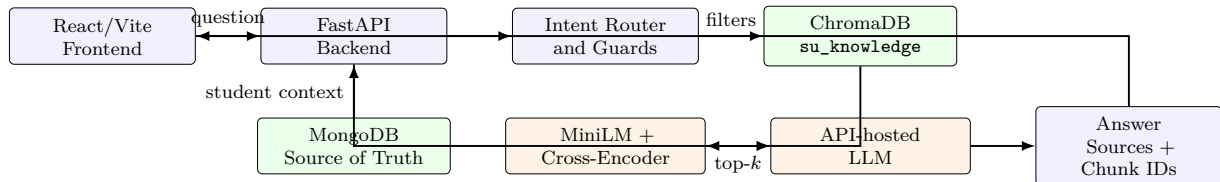


Figure 1: Runtime architecture of SU-GPT. The measured local configuration uses MongoDB via Docker and ChromaDB persistence; the storage contract is compatible with a later managed deployment.

3.1 Frontend and Backend

The frontend is implemented with React and Vite. It contains the chat interface, authentication view, course-selection UI, and document-upload controls. The backend is implemented with FastAPI. The `/ask/` endpoint receives a question and optional username, resolves intent, retrieves or constructs context, loads user course history, builds the prompt, invokes the LLM, and returns the generated answer with sources.

The LLM provider is configurable. The measured local run used `LLM_PROVIDER=mistral` with `mistral-small-latest`; the project also contains Groq/Llama configuration for deployments that use Groq-hosted Llama 3.3 70B. The report therefore describes the architecture as API-hosted LLM-ready rather than tying all measured results to one provider.

3.2 MongoDB Source of Truth

MongoDB stores structured application state, including users, course records, user-course selections, source documents, ingestion jobs, upload batches, reviews, exams, and embedding-cache metadata. In local experiments, MongoDB runs via Docker; the schema is compatible with a managed MongoDB Atlas deployment. ChromaDB is not the source of truth. It is a reproducible vector index that can be rebuilt from MongoDB source metadata and stored files.

3.3 ChromaDB Vector Index

ChromaDB stores retrieval-ready text, vector embeddings, metadata, and stable chunk identifiers in the `su_knowledge` collection. The active local index contains 229,631 chunks. All indexed chunks in the current benchmark-relevant corpus use `documentType=course`; review and exam corpora are intentionally empty in this phase.

Table 1: Principal system components in the current implementation.

Component	Technology / Current Role
Frontend	React + Vite
Backend	FastAPI + Python
Operational database	MongoDB, local Docker in measured run; Atlas-ready schema
Vector database	ChromaDB, <code>su_knowledge</code> collection
Embedding model	<code>sentence-transformers/all-MiniLM-L12-v2</code>
Reranker	<code>cross-encoder/ms-marco-MiniLM-L-6-v2</code>
Generator	API-hosted LLM; measured local run used Mistral
Evaluation outputs	JSONL/JSON/CSV under <code>outputs/</code>

4 Data and Preprocessing

4.1 Data Sources

The benchmark-relevant data comes from JSONL course and curriculum records plus uploaded course-material sources. Table 2 lists the main structured sources.

Table 2: Main data sources and roles.

Path	Role
<code>data/all_coursepage_info.jsonl</code>	Course titles, descriptions, credits, prerequisites, corequisites, source URLs.
<code>data/{term}/{program}.jsonl</code>	Program-term classification snapshots, SU credits, ECTS, engineering ECTS, basic-science ECTS.
<code>data/degree_requirements/CS.jsonl</code>	Detailed BSCS requirements for curriculum term 202101.
<code>data/schedule/202502.jsonl</code>	Sections, CRNs, instructors, meeting times, rooms, and components.
<code>data/graduation_requirements.jsonl</code>	General graduation requirement records.
<code>sources/Module*</code>	CS455/555 course-material documents used for lecture-content retrieval.

4.2 Ingestion

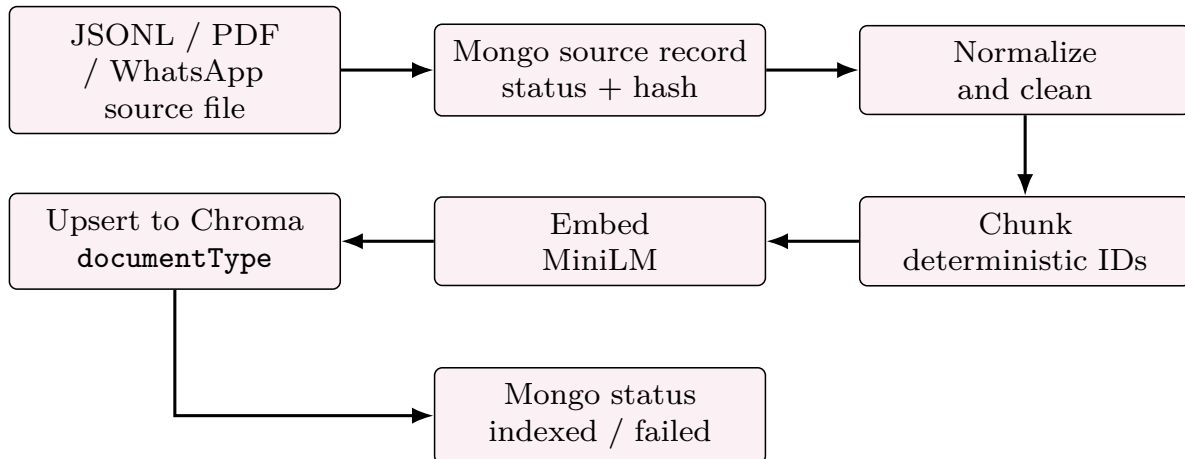


Figure 2: Unified ingestion backbone. MongoDB stores durable source-of-truth metadata; ChromaDB is treated as a reproducible index.

Structured rows are converted into explanatory information cards. A course-catalog row becomes a short card with course code, title, credits, prerequisites, corequisites, description, and source URL. A schedule row becomes a card with term, CRN, instructor, meeting time, room, and component. Every chunk receives a deterministic ID with the shape:

`sourceId:chunkIndex:contentHashPrefix`

The measured benchmark contains verified chunk IDs. For example:

`course:5a6a448569ef18c6d8137810:150:5f9083e5db8d5499`

These stable IDs allow retrieval metrics to be computed at the chunk level.

5 Question-Answering Pipeline

5.1 Intent Routing and Source Availability

The system first predicts or resolves an intent. The current routes include graduation status, course recommendation, study plan, major selection, specialization, course detail, review, exam, and general fallback. When local evidence is available, the answer should be grounded in retrieved context. When local evidence is intentionally absent, as for review, exam, and syllabus-specific corpora in this phase, the system should state that it lacks a local source and provide cautious general guidance instead of pretending to have retrieved evidence. For example, a missing-review answer can advise the student to check course groups or email the instructor with a short background and question, while clearly marking that no local review archive was used.

5.2 Retrieval and Generation

The default final context size is $k = 6$. Dense retrieval uses Chroma similarity search. Hybrid rerank combines BM25 and dense candidates, fuses them with RRF, and reranks with a cross-encoder. The final response includes both `sources` and `source_chunk_ids`.

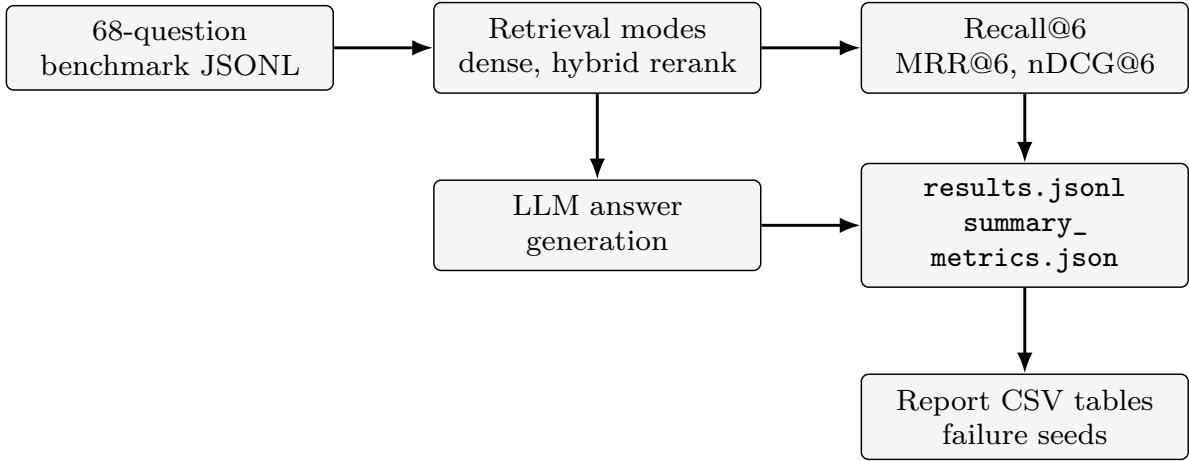


Figure 3: Evaluation pipeline implemented for the current measured pilot run. Full 68-question, six-mode execution is supported by the CLI but remains to be run before final numeric claims are frozen.

6 Evaluation Methodology

6.1 Benchmark

The current benchmark file contains 68 grounded questions. Each JSONL row includes an ID, question, language, question type, expected intent, expected document type, answerability flag, reference answer, expected sources, expected chunk IDs, and notes. Every expected chunk ID was verified against the live `su_knowledge` collection before evaluation.

Table 3: Benchmark composition from `data/benchmark/questions.jsonl`.

Document type	Intent / answerability group	Count
course	<code>ders_ayrintisi</code> , answerable	20
course	<code>diger</code> , answerable	20
course	<code>diger</code> , unanswerable	16
course	<code>mezuniyet_durumu</code> , answerable	7
exam	<code>exam</code> , unanswerable	2
review	<code>review</code> , unanswerable	2
course	<code>ders_ayrintisi</code> , unanswerable	1
Total		68

6.2 Metrics

Retrieval is measured with Recall@K, MRR@K, and nDCG@K. Chunk-level matching is used when `expected_chunk_ids` exist. If a row has no expected chunk IDs, the evaluator falls back to source-level matching using `expected_sources`. For the measured pilot run, $K = 6$ because the runner defaulted to `top_k=6`.

$$\text{Recall@K} = \frac{|\text{Retrieved}_{1:K} \cap \text{Gold}|}{|\text{Gold}|}, \quad (1)$$

$$\text{MRR@K} = \frac{1}{\text{rank of first relevant hit}}, \quad (2)$$

$$\text{nDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}}. \quad (3)$$

6.3 Measured Run

The current measured run is stored under `outputs/evaluation_runs/20260607_231717`. It evaluated the first 10 benchmark questions with two modes: dense and hybrid rerank. LLM generation was enabled. This gives 20 result rows and 120 retrieved-chunk rows.

Pending measured result: Before final submission, run the full benchmark with all six modes, or explicitly state that the project reports a pilot evaluation only. The CLI already supports this via `server/evaluation/run_evaluation.py`.

7 Results

7.1 Retrieval Metrics

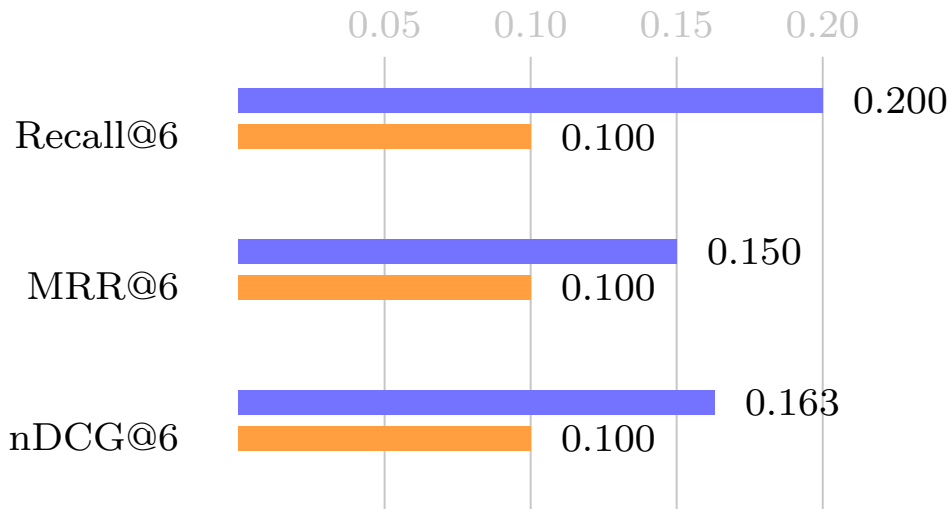


Figure 4: Measured pilot retrieval metrics on the first 10 benchmark questions. Blue bars show dense retrieval; orange bars show hybrid rerank. The run used $k = 6$ and generated LLM answers.

Table 4: Measured pilot retrieval results on the first 10 questions, $K = 6$.

Mode	Rows	Recall@6	MRR@6	nDCG@6	Chunk hit rate
Dense	10	0.200	0.150	0.163	0.200
Hybrid rerank	10	0.100	0.100	0.100	0.100

On this small pilot slice, dense retrieval outperformed hybrid rerank. This should not be interpreted as a universal conclusion, because the sample is only the first 10 benchmark rows and the hybrid path paid a substantial retrieval/reranking cost. The result is nevertheless useful: it shows that hybrid and rerank logic can fail if the right chunk is absent from the candidate set or if near-identical template rows dominate the fused ranking.

7.2 Latency and Efficiency

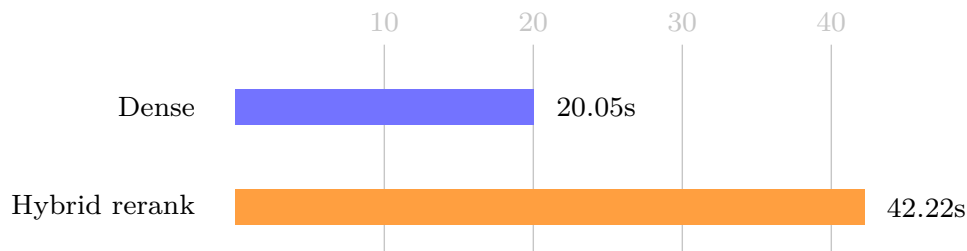


Figure 5: Mean end-to-end latency for the measured pilot run, including retrieval and generation. Blue indicates dense retrieval and orange indicates hybrid rerank. Hybrid rerank was slower primarily because retrieval and reranking dominate the runtime.

Table 5: Measured pilot efficiency metrics. Values are averages across the 10 questions per mode.

Mode	Total latency	Retrieval	Rerank	Generation
Dense	20.05s	18.36s	0.00s	1.69s
Hybrid rerank	42.22s	36.60s	3.35s	2.25s

The primary bottleneck is retrieval over the large local Chroma/BM25 corpus, not LLM generation. A previous interactive `/ask/` optimization reduced common structured academic queries to below 10 seconds by bypassing broad vector search for intents that can be answered from structured catalog files. The benchmark runner, however, intentionally measures retrieval modes directly, so its latency exposes the raw retrieval cost.

7.3 Answer Metrics

Pending measured result: Manual answer correctness, citation correctness, faithfulness, answer relevancy, and hallucination labels have not been filled. The generated `outputs/tables/answer_metrics.csv` keeps these fields blank rather than fabricating scores. Fill them after a manual grading pass over `results.jsonl`.

8 Failure Analysis

The Section 6 tooling currently contains two real seed failure cases in:

`outputs/failure_analysis/failure_cases.csv`

Both are retrieval misses under hybrid rerank with $K = 6$.

Table 6: Seed failure cases from observed runs.

ID	Question type	Error type	Observed root cause
q008	CS412 schedule lookup	retrieval miss	The verified CS412 schedule chunk was absent from top-6; near-identical 202502 schedule chunks for other courses out-ranked the exact course row.
q020	BERT position-embedding slide lookup	retrieval miss	The gold slide 17 chunk was very short; neighboring/richer chunks such as slide 16 ranked above the exact title-only slide.

8.1 Course-Code and Schedule Ambiguity

Schedule records are highly template-like. A query mentioning **CS412** can retrieve other course sections from the same term because the surrounding text is nearly identical. Exact course-code extraction and normalized metadata filtering should run before dense/BM25 fusion for schedule questions.

8.2 Short Chunk Weakness

The q020 miss shows that short slide-title chunks can be under-ranked. A title such as “Position Embeddings (Intuition)” contains too little context compared with neighboring slides. A practical fix is to merge very short slide-title chunks with adjacent slide text or add title/slide-number boosts when a question asks about a specific slide.

8.3 Missing Review, Exam, and Syllabus Sources

The project currently has zero review chunks and zero exam chunks by design. The assistant must not claim retrieved review or exam evidence. The appropriate behavior is to say that the project is still under development for that source type, give cautious general guidance, and suggest checking official materials, student groups, or emailing the instructor. This policy should be evaluated separately from course-catalog retrieval.

9 Ablations

The Section 6 configuration file defines a future grid over chunk sizes, chunk overlaps, top- k values, retrieval modes, and prompt strategies. Only a tiny quick ablation has been run so far: dense retrieval with top- $k = 3$ over one benchmark question, which produced zero hit metrics and an average latency of 16.58 seconds. This is a smoke test of the ablation runner, not a scientific ablation result.

Pending measured result: Run the full retrieval-only ablation grid after the final benchmark run. Prompt strategies `few_shot` and `expert_routed` should remain skipped until Section 4 prompt/router modules exist.

10 Limitations and Future Work

The current system is not yet reliable as an official multi-program degree advisor. Detailed official degree requirements are available only for the CS/BSCS 202101 curriculum in the current structured file. User profiles need major, degree code, admission term, curriculum term, current term, minor codes, course status, transfer status, repeated-course handling, and withdrawal status.

Academic arithmetic should move out of the LLM and into a deterministic degree-audit module. The LLM should explain the computed result, not decide category allocation. Future degree-audit logic should apply equivalencies, choice pools, overflow rules, repeated-course handling, total SU credits, ECTS, engineering ECTS, basic-science ECTS, and GPA constraints.

Immediate retrieval improvements include:

- normalize compact and spaced course identifiers at ingestion and query time;
- add exact course-code filters or boosts for schedule and catalog queries;
- enrich very short slide-title chunks with neighboring context;
- separate public curriculum sources from private student-specific degree evaluations;
- complete the full 68-question, six-mode benchmark run;
- manually grade answer correctness and citation correctness;
- expand unsupported review/exam/syllabus behavior tests so the assistant gives useful caveated advice without pretending to have local evidence.

11 Reproducibility

Backend setup:

```
cd server
python -m venv .venv
.venv/bin/pip install -r requirements.txt
docker compose up -d mongo
.venv/bin/python -m uvicorn main:app --host 127.0.0.1 --port 8000
```

Frontend setup:

```
cd frontend
npm install
npm run dev
```

Evaluation runner:

```
server/.venv/bin/python server/evaluation/run_evaluation.py \
--modes dense hybrid_rerank --limit 10
```

The reported pilot metrics were produced from `outputs/evaluation_runs/20260607_231717`. Report tables were generated under `outputs/tables/`. The current run used $K = 6$.

12 Ethical Considerations and AI Disclosure

Academic advice can affect important student decisions. SU-GPT should not claim that a student can graduate unless the result uses the correct official curriculum and a complete student record. Answers must remain advisory and direct students to official Degree Evaluation and advisors for final confirmation.

Course histories and uploaded academic records are personal data. Future versions should enforce authentication, ownership filters, access control, and deletion policies. Private student documents must never be retrievable by another user.

Generative AI tools were used as development assistants for codebase analysis, debugging, evaluation tooling, documentation, and report drafting. The project members remain responsible for reviewing code, running experiments, checking source data, and validating reported results. Numerical values in this paper are copied from measured local output files and are not fabricated.

13 Conclusion

SU-GPT demonstrates a working retrieval-augmented academic advising application with a modern frontend, FastAPI backend, MongoDB source-of-truth model, ChromaDB vector index, deterministic ingestion, multiple retrieval modes, and a reproducible benchmark runner. The current measured pilot run is deliberately modest: 10 questions, two modes, and $K = 6$. It still reveals important engineering facts. Dense retrieval reached Recall@6 of 0.20, while hybrid rerank reached 0.10 and incurred higher latency. The failures are mainly retrieval failures, not generation failures: the correct evidence often fails to reach the prompt.

The next step is not to make the model more fluent; it is to make retrieval more curriculum-aware. Exact course-code normalization, schedule-specific filters, short-chunk enrichment, deterministic degree auditing, and full benchmark execution are the most important steps before claiming official advising reliability.

References

- [1] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *NeurIPS*, 2020.
- [2] V. Karpukhin et al., “Dense Passage Retrieval for Open-Domain Question Answering,” *EMNLP*, 2020.
- [3] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, “REALM: Retrieval-Augmented Language Model Pre-Training,” *ICML*, 2020.
- [4] G. Izacard and E. Grave, “Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering,” *EACL*, 2021.
- [5] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” *EMNLP-IJCNLP*, 2019.
- [6] R. Nogueira and K. Cho, “Passage Re-ranking with BERT,” arXiv:1901.04085, 2019.
- [7] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Foundations and Trends in Information Retrieval*, 2009.
- [8] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods,” *SIGIR*, 2009.
- [9] O. Khattab and M. Zaharia, “ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT,” *SIGIR*, 2020.
- [10] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” *IEEE TPAMI*, 2018.
- [11] J. Johnson, M. Douze, and H. Jégou, “Billion-Scale Similarity Search with GPUs,” *IEEE Transactions on Big Data*, 2019.
- [12] A. Vaswani et al., “Attention Is All You Need,” *NeurIPS*, 2017.
- [13] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *NAACL*, 2019.
- [14] T. Wolf et al., “Transformers: State-of-the-Art Natural Language Processing,” *EMNLP: System Demonstrations*, 2020.
- [15] J. Lin, R. Nogueira, and A. Yates, “Pretrained Transformers for Text Ranking: BERT and Beyond,” *Synthesis Lectures on Human Language Technologies*, 2021.
- [16] N. F. Liu et al., “Lost in the Middle: How Language Models Use Long Contexts,” *TACL*, 2024.
- [17] OpenAI, “GPT-4 Technical Report,” arXiv:2303.08774, 2023.
- [18] LangChain Documentation, <https://python.langchain.com/>.
- [19] Chroma Documentation, <https://docs.trychroma.com/>.
- [20] MongoDB Atlas Documentation, <https://www.mongodb.com/docs/atlas/>.
- [21] Sentence Transformers Documentation, <https://www.sbert.net/>.